PRACTICAL AI

Build your first AI-written tool

A step-by-step guide for beginners — no coding experience needed, just a task worth automating

> You don't need to be a programmer to get AI to build you a small tool that saves you time every week. You do need to follow the steps in the right order, or it can feel confusing fast.
>
> This tutorial takes you through the whole process, in the order it actually needs to happen: deciding whether it's worth doing, getting the code, setting your computer up to run it, testing it safely, and running it for real.

QUICK ANSWER

Pick one repetitive task, describe it clearly to an AI tool, and ask it to write you a short Python program. Install Python once, install whatever extra bits your program needs, test it on fake data first, then run it on the real thing. The whole first attempt usually takes under an hour, most of it waiting for downloads.

BEFORE YOU START: WHAT YOU'LL NEED

- A computer (Windows or Mac)
- Access to an AI tool such as ChatGPT or Claude
- One task you do by hand, at least once a week
- About 30–60 minutes, most of it for downloads and testing

You won't need to write a single line of code yourself. You will need to copy, paste, click, and read carefully.

Stage 1: Decide whether this is worth building

Not every repetitive task is worth turning into a tool. Before you spend any time on this, ask yourself three questions.

Do the steps stay the same each time, and only the details change? If yes, that's a good sign. If the task needs judgement or changes shape every time, code probably isn't the right fit, and using AI directly each time is the better choice.

Do you do this at least once a week? If it's something you do once a year, building a tool isn't worth the effort. Save the time for something that repeats often.

Would getting it wrong cause real damage? If a mistake here could cost you money, upset a customer, or breach a data protection rule, you'll need someone with a bit more technical confidence to check the finished tool before you rely on it. That doesn't rule it out. It just means don't skip the testing steps later in this guide.

TIP — IF YOU'RE STILL NOT SURE, ASK AI TO DECIDE WITH YOU

Paste this into your AI tool and describe your task where indicated:

```
I have a task I do regularly: [describe what you do, how often,
and what files or information are involved].

Should I:
A) Keep using AI directly each time
B) Build a simple, reusable tool
C) Use a mix of both

Tell me which you'd recommend and why, and flag any data
protection risks I should think about.
```

Stage 2: Pick your task and write it down properly

Choose one task. Not five. One.

Here are a few examples to give you a feel for what a "good starter task" looks like:

Business	Task	What's involved
Café	Weekly stock order	Check what's running low, create a reorder list
Retail shop	Weekly sales summary	Pull totals from a spreadsheet, spot best-sellers
Hair salon	Appointment revenue	Count appointments by service, work out the total

Accountancy practice	Client data clean-up	Remove duplicate entries, standardise formatting
Marketing agency	Monthly campaign report	Pull stats together, calculate averages

Once you've picked your task, write a short, specific description of it. This step matters more than people expect, because a vague description gets you vague code.

Fill this in:

```
Every [how often], I [what you currently do by hand].

The input is [what file or information you start with, and
what's in it – e.g. a CSV with columns: date, customer, price].

I want the tool to:
- [step]
- [step]

The output should be [what you want to end up with].

Don't [anything you specifically don't want it to do].
```

Example, filled in:

```
Every Monday, I download a CSV file from my till system.

The input is a CSV with columns: date, customer, product,
price, status.

I want the tool to:
- Remove any cancelled orders
- Add up total sales by product
- Create a short summary

The output should be a new spreadsheet with that summary.

Don't change or overwrite the original file. Tell me if any
columns are missing.
```

Stage 3: Set your computer up (you only do this once)

Do this before you ask AI for any code, so everything's ready when the code arrives.

You need a free program called **Python** installed on your computer. It's what actually runs the code AI writes for you.

Windows:

1. Go to python.org and click the yellow "Download Python" button.
2. Open the file you downloaded.
3. Tick the box that says **"Add Python to PATH"**. This step is easy to miss and causes most of the problems people run into later.
4. Click "Install Now" and wait for it to finish.

Mac:

1. Open Terminal (search for it with Spotlight).
2. Type `python3 --version` and press Enter.
3. If you see a version number, Python's already there. If not, download it from python.org.

Check it worked. First, you'll need to open the right programme:

On Windows, to open Command Prompt:

1. Click the Start button (or press the Windows key on your keyboard).
2. Type `cmd`.
3. Click **Command Prompt** when it appears in the search results (it has a black icon).
4. A black window with white text will open. That's it, that's Command Prompt.

On Mac, to open Terminal:

1. Press **Cmd + Space** to open Spotlight search.
2. Type `Terminal`.
3. Press Enter, or click **Terminal** when it appears.
4. A window will open, usually with a white or black background and some text in it. That's Terminal.

You'll use this same window (or a new one, opened the same way) throughout the rest of this tutorial, so it's worth knowing how to get back to it.

Once it's open, type `python --version` (Mac: `python3 --version`) and press Enter. You should see something like "Python 3.12.1".

> **TIP — NOT SURE IF YOU'VE TYPED IT RIGHT?**
>
> Command Prompt and Terminal are fussy about exact spelling and spacing, but forgiving about capital letters in most commands. If nothing seems to happen after pressing Enter, check for typos first, that's the most common cause.

> **CAUTION — DON'T SKIP THE "ADD TO PATH" STEP ON WINDOWS**
>
> If you miss this tickbox during installation, your computer won't recognise Python commands later, and you'll see confusing error messages. If that happens, it's usually quicker to uninstall and reinstall, ticking the box, than to fix it after the fact.

Stage 4: Ask AI to write your code

Now open your AI tool and paste in the specification you wrote in Stage 2. Add this to the end of it:

> "*Write a Python program that does this. Explain what each part does in plain English. Tell me exactly what I need to install to run it, and how to test it safely before using it on real data.*"

AI will give you back four things: the code itself, a plain-English explanation, a list of anything extra you need to install (called "dependencies" — more on those in a moment), and instructions for testing it.

Read the explanation before doing anything else. You're checking two things: does this match what you actually asked for, and does anything about it worry you, such as sending your data somewhere you don't expect?

You don't need to understand code to check it. You need to understand your own task well enough to spot when the explanation doesn't match it.

Stage 5: Install what your code needs

A "dependency" is just an extra bit of free code your program borrows to do something Python can't do by itself, most commonly, reading spreadsheets.

AI will usually tell you what to install. It normally looks like this:

```
pip install pandas openpyxl
```

(On a Mac, if that doesn't work, try `pip3` instead of `pip`.)

Type that exact line into Terminal (Mac) or Command Prompt (Windows) and press Enter. You'll see a lot of text scroll past, then a message saying something was "Successfully installed". That's it, done, and you won't need to do it again for that program.

Common dependencies, so you know roughly what to expect:

- CSV or Excel files → `pandas openpyxl`
- PDF files → `PyPDF2`
- Images → `Pillow`
- Fetching data from a website or service → `requests`

If you see "pip is not recognised": type `python -m pip install pandas openpyxl` instead (Mac: `python3 -m pip install pandas openpyxl`).

Stage 6: Save the code as a file

1. Open a plain text editor: Notepad on Windows, TextEdit on Mac (switch it to Plain Text first), or download the free VS Code if you'd like something a bit friendlier.
2. Copy the code AI gave you and paste it in.
3. Save it with a name that ends in `.py`, for example `sales-report.py`. On Windows, make sure "Save as type" is set to "All Files", not `.txt`, or it'll save with the wrong file ending.
4. Save it somewhere easy to find, like a new folder on your Desktop called `ai-tools`.

Stage 7: Test it on fake data first

Never point a new script at your real, live data first. Create a small test file instead, with a few made-up rows in the same format your real file uses.

Example test file (save as `test-data.csv`, in the same folder as your script):

```
date,customer,product,price,status
2026-01-15,John Smith,Widget,25.00,completed
2026-01-15,Jane Doe,Gadget,45.00,cancelled
2026-01-16,Bob Jones,Widget,25.00,completed
```

Now run it. The easiest way: find your `.py` file and drag it onto the open Terminal or Command Prompt window, then press Enter.

If that doesn't work, type this instead, from the folder your file's saved in:

- Windows: `python my-tool.py`
- Mac: `python3 my-tool.py`

Check the output carefully. Are the numbers right? Is anything missing? If something's off, go back to AI, describe exactly what happened and what you expected instead, and ask it to fix the code. Replace the old version and try again.

Stage 8: When something goes wrong

This happens to almost everyone the first time, and it's normal. Here's what the most common messages actually mean.

- **"No module named 'pandas'"** → you haven't installed a dependency yet. Go back to Stage 5.
- **"File not found"** → your script is looking for a file that isn't in the same folder, or the name doesn't match exactly.
- **"Permission denied"** → close the file if it's open in another program, and try again.
- **"pip is not recognised"** → use `python -m pip install ...` instead.
- **Anything else** → copy the exact error message and paste it into your AI tool with: "I ran the Python program you wrote and got this error. What does it mean and how do I fix it?"

Stage 9: Run it on your real data

Once the test version works properly:

1. Make a backup copy of your real data file first.
2. Run the script on a copy of your real file, not the original.
3. Check the output again, the same way you did with the test data.
4. If it all looks right, you can start using it with confidence.

> **CAUTION — CHECK IN ON IT OCCASIONALLY**
>
> A script that works today can quietly start giving wrong results later, if a column gets renamed, a file format changes, or something updates on your computer. Get into the habit of spot-checking the output every so often rather than assuming it'll always be correct.

Optional: turn it into a simple web app

If you end up using your tool regularly, you can make it easier for yourself or your team to run, without needing the command line at all.

Ask AI: "Rewrite this as a Streamlit app." Then:

1. Install Streamlit: `pip install streamlit`
2. Save the new code, for example as `my-tool-app.py`.
3. Run it with: `streamlit run my-tool-app.py`

This opens a simple page in your web browser with buttons and boxes, so anyone on your team can use it without touching any code.

Common mistakes to avoid

- **Running it on real data first** — always test on a fake sample.
- **Not reading the output** — check the results before you trust them.
- **Deleting or overwriting the original file** — keep a backup, always.
- **Assuming it'll keep working forever** — check in on it now and then.
- **Not asking what happens to your data** — check this with AI before you run anything that touches customer or financial information.

Quick reference: AI, code, or both?

Use AI directly when: the task needs judgement or creativity, the input's different every time, or it only comes up occasionally.

Build code when: you do it at least weekly, the steps don't change, and it's mostly moving, cleaning or calculating data.

Use a hybrid when: code can handle the repetitive part, and AI is only needed to interpret or decide on the tricky bits.

Try this next

Copy this into your AI tool right now:

> "*I do this task every week: [describe it]. Could a simple Python script do it instead? If so, write me one. Explain what each part does, tell me exactly what to install, and tell me what could go wrong.*"

You don't have to use whatever comes back. Just see whether it looks like something worth trying.

Sources

[1] Python Software Foundation — Python Documentation
Trust: ★★★★★ | Category: Technology organisations

Official reference for installing and running Python.

[2] Streamlit — Documentation
Trust: ★★★★★ | Category: Technology organisations

Official reference for turning a Python script into a simple web app.

[3] National Cyber Security Centre — Guidance on AI-assisted software development
Trust: ★★★★★ | Category: Regulators and official UK sources

Used for the principle that oversight should match the sensitivity of the task and the consequences of something going wrong.

[4] Information Commissioner's Office — Data protection guidance for small organisations
Trust: ★★★★★ | Category: Regulators and official UK sources

Basis for the note that data protection responsibility stays with you, whether a person or a script is handling the data.

Evidence note

AI-written code can genuinely save small business owners time on repetitive tasks, but it isn't a substitute for professional software development where mistakes would be costly or where sensitive data is involved. How much checking and testing a tool needs should match how much damage a mistake could cause, not a fixed rule for every task. Data protection responsibility stays with you as the data controller, whatever is doing the processing, a person or a script.

This tutorial is practical guidance, not technical or legal advice.